



The
**University
of Lahore**

TOOLS AND TECHNIQUES

DATA PREPROCESSING AND ANALYSIS

*Application of all techniques learned in the course
on the selected dataset with clear explanation
of how and why each technique was applied
and the top results to demonstrate its impact.*



SUBMITTED BY:
AYESHA NAFIS
70189383



SUBJECT:
TOOLS AND TECHNIQUES



SUBMITTING TO:
Dr Iftikhar Ahmad



DATE:
16 May, 2026

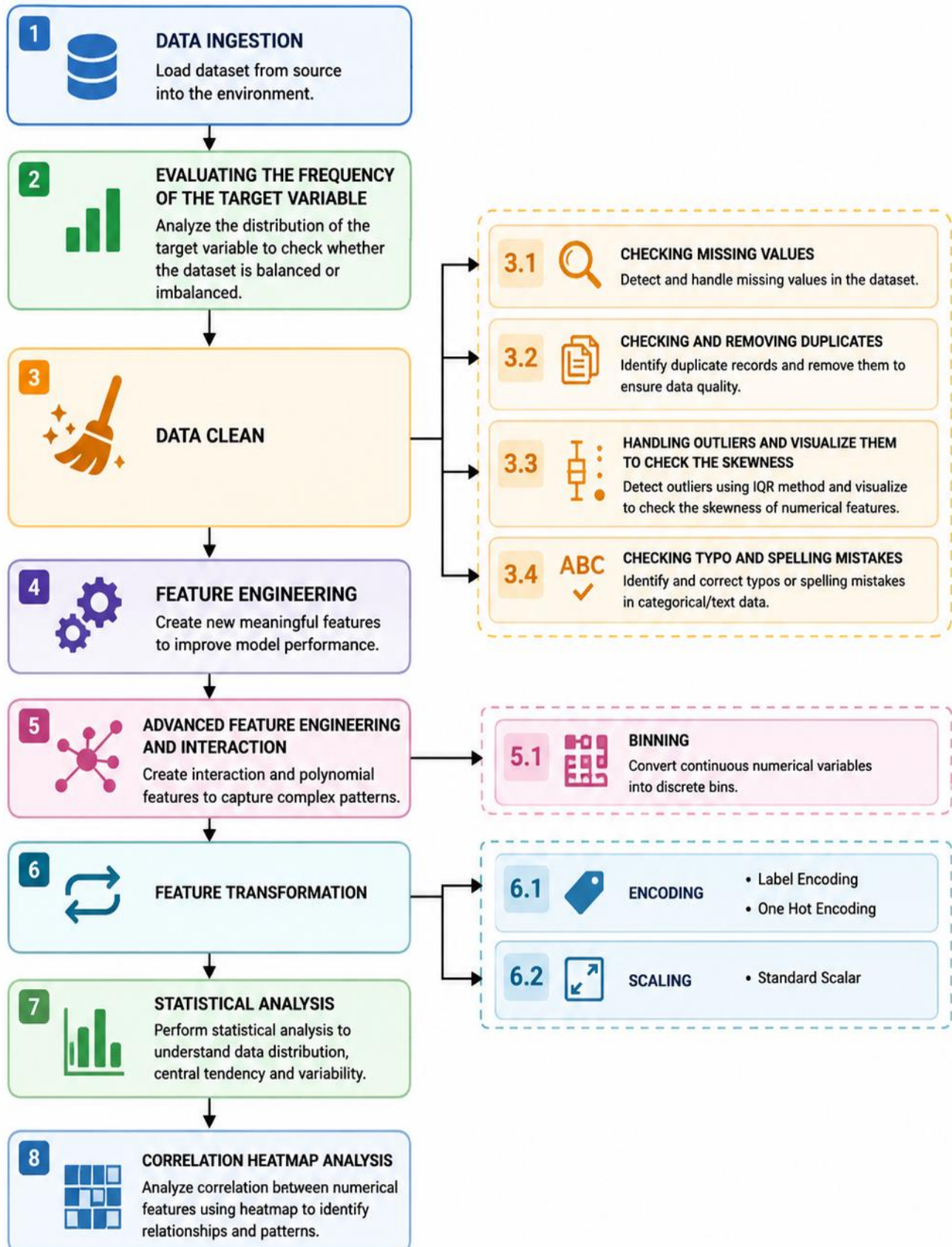


“Turning Data into Insights, Insights into Impact.”

Summary of the Assignment

EDA PROCESS ARCHITECTURE

From Data Ingestion to Data Preprocessing



1. Introduction

This project was developed using Git and Google Colab to maintain proper version control and project management. Git was used to clone and manage the project repository, which helped in organizing files and tracking project progress efficiently. The project workflow included dataset loading, preprocessing, feature engineering, encoding, scaling, and data preparation techniques taught during the course.

Git Repository Integration

Technique Applied: Git Version Control

Why this technique was applied:

Git was used to manage the project files, maintain code versions, and clone the project repository from GitHub.

This project was performed to apply all the techniques taught in the course on a selected dataset. The selected dataset is based on online shoppers' purchasing behavior. The main objective of this project is to preprocess the data, clean the dataset, handle outliers, perform feature engineering, apply encoding and scaling techniques, and prepare the dataset for machine learning.

The target variable of this dataset is **Revenue**, which represents whether a customer generated revenue or not.

Dataset file used in this project: CA1.csv

Git Files

Repo Name: Tool-and-Tech-70189383

Repo Link: <https://github.com/ayeshanafees48/Tools-and-Tech-70189383>

Files Name:

- CA.csv
- Encoder.py
- Graph_Visualize.py
- Scaler.py
- Final Colab Code File

2. Loading the Dataset

Technique Applied: Dataset Loading using Pandas

Why this technique was applied:

The dataset needed to be imported into Python so that preprocessing and analysis techniques could be performed.

Code:

```
# Load Dataset
df = pd.read_csv("CA1.csv")
print("-----")
print("Original Dataset Shape:", df.shape)
print("-----")
```

Figure 1: Loading Dataset

Results:

```
... -----
Original Dataset Shape: (12330, 18)
-----
```

Figure 2: Original Data Shape

3. Evaluating Frequency of Target Variable

Technique Applied: Frequency Distribution

Why this technique was applied:

The target variable was analyzed to check whether the dataset is balanced or imbalanced.

```
print(df['Revenue'].value_counts())
# Evaluating Frequency of Dataset
df = pd.read_csv("CA1.csv")
print(df['Revenue'].value_counts(normalize=True) * 100)

Revenue
False    10422
True      1908
Name: count, dtype: int64
Revenue
False     84.525547
True     15.474453
Name: proportion, dtype: float64
```

Figure 3: Evaluating Frequency of Target Variable

Result:

The dataset was found to be imbalanced because the number of False values was much greater than the number of True values.

Since the dataset is imbalanced, balancing techniques such as Oversampling, Undersampling, and smoth may be required.

4. Data Cleaning

4.1 Checking Missing Values

Technique Applied: Missing Value Detection

Why this technique was applied:

Missing values can reduce model performance and cause errors during training.

Code:

```
# Check missing values
print("Missing Values in Each Column:")
print(df.isnull().sum())
# Percentage of missing values
print("\nMissing Values Percentage:")
print((df.isnull().sum() / len(df)) * 100)
```

Figure 4: Checking the Missing Values

Result:

No significant missing values were found in the dataset.

```
Missing Values in Each Column:
Administrative          0
Administrative_Duration 0
Informational           0
Informational_Duration  0
ProductRelated          0
ProductRelated_Duration 0
BounceRates             0
ExitRates               0
PageValues              0
SpecialDay              0
Month                   0
OperatingSystems        0
Browser                 0
Region                  0
TrafficType             0
VisitorType             0
Weekend                 0
Revenue                 0
dtype: int64
```

Figure 5: Results of Missing Values

4.2 Checking Duplicate Records

Technique Applied: Duplicate Detection and Removal

Why this technique was applied:

Duplicate rows can negatively affect analysis and machine learning model accuracy.

Code:

```
duplicate_Count = df.duplicated().sum()
print("Duplicate Count (before):", duplicate_Count)
if duplicate_Count > 0:
    duplicate_Rows = df[df.duplicated(keep=False)]
    print("\nDuplicate Rows are:")
    print(duplicate_Rows)
    df = df.drop_duplicates()
    new_count = df.duplicated().sum()
    print("\nDuplicates removed successfully!")
    print("Duplicate Count (after):", new_count)
    print("New shape of dataset:", df.shape)
else:
    print("\nThere is no duplicate in dataset (even after cleaning)")
```

Figure 6: Checking Duplicates

Results:

Found 125 rows duplicates and removes successfully.

Duplicate Count (before): 125				
Duplicate Rows are:				
	Administrative	Administrative_Duration	Informational	\
85	0	0.0	0	
132	0	0.0	0	
158	0	0.0	0	
159	0	0.0	0	
178	0	0.0	0	
...	...	...	...	
11934	0	0.0	0	
11938	0	0.0	0	
12159	0	0.0	0	
12180	0	0.0	0	
12185	0	0.0	0	
	Informational_Duration	ProductRelated	ProductRelated_Duration	\
85	0.0	1	0.0	
132	0.0	1	0.0	
158	0.0	1	0.0	
159	0.0	1	0.0	
178	0.0	1	0.0	
...	...	...	...	
11934	0.0	1	0.0	
11938	0.0	1	0.0	
12159	0.0	1	0.0	
12180	0.0	1	0.0	
12185	0.0	1	0.0	

Figure 7: Duplicates Result

	BounceRates	ExitRates	PageValues	SpecialDay	Month	OperatingSystems	\
85	0.2	0.2	0.0	0.0	Feb	1	
132	0.2	0.2	0.0	0.0	Feb	3	
158	0.2	0.2	0.0	0.0	Feb	1	
159	0.2	0.2	0.0	0.0	Feb	3	
178	0.2	0.2	0.0	0.0	Feb	3	
...	...	...	...	...	...	...	
11934	0.2	0.2	0.0	0.0	Dec	1	
11938	0.2	0.2	0.0	0.0	Dec	1	
12159	0.2	0.2	0.0	0.0	Dec	1	
12180	0.2	0.2	0.0	0.0	Dec	1	
12185	0.2	0.2	0.0	0.0	Dec	8	
	Browser	Region	TrafficType	VisitorType	Weekend	Revenue	
85	1	1	3	Returning_Visitor	False	False	
132	2	3	3	Returning_Visitor	False	False	
158	1	1	3	Returning_Visitor	False	False	
159	2	3	3	Returning_Visitor	False	False	
178	2	3	3	Returning_Visitor	False	False	
...	...	...	...	...	...	...	
11934	1	1	2	New_Visitor	False	False	
11938	1	4	1	Returning_Visitor	True	False	
12159	1	1	3	Returning_Visitor	False	False	
12180	13	9	20	Returning_Visitor	False	False	
12185	13	9	20	Other	False	False	
[201 rows x 18 columns]							
Duplicates removed successfully!							

Figure 8: Removal Dataset

4.3 Checking Typo and Spelling Errors

Technique Applied:

Why this technique was applied:

Categorical columns may contain spelling mistakes or inconsistent values.

Code:

```
valid_months = ['jan', 'feb', 'mar', 'apr', 'may', 'june', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec']
df['Month'] = df['Month'].str.lower().str.strip()
invalid_months = df[~df['Month'].isin(valid_months)][['Month']].unique()
print("Invalid Month Values:", invalid_months)
valid_types = ['returning_visitor', 'new_visitor', 'other']
df['VisitorType'] = df['VisitorType'].str.lower().str.strip()
invalid_types = df[~df['VisitorType'].isin(valid_types)][['VisitorType']].unique()
print("Invalid VisitorType Values:", invalid_types)
```

Figure 9: Typo Error

Results:

```
... Invalid Month Values: []
Invalid VisitorType Values: []
```

Figure 10: Result of Typo Error

4.4 Standardize date format

Technique Applied: Not Valid because in my data set date column is not exist.

5. Handling Outliers

Technique Applied: IQR Method

Why this technique was applied:

Outliers can distort statistical analysis and reduce machine learning performance.

Code:

```
import numpy as np
numerical_cols = [
    "Administrative",
    "Administrative_Duration",
    "Informational",
    "Informational_Duration",
    "ProductRelated",
    "ProductRelated_Duration",
    "BounceRates",
    "ExitRates",
    "PageValues",
    "SpecialDay"
]
for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(col, "Outliers:", len(outliers))
```

Figure 11: Evaluating Outliers using IQR

Result:

Outliers were identified in multiple numerical columns.

```
Administrative Outliers: 404
Administrative_Duration Outliers: 1149
Informational Outliers: 2631
Informational_Duration Outliers: 2405
ProductRelated Outliers: 1007
ProductRelated_Duration Outliers: 951
BounceRates Outliers: 1428
ExitRates Outliers: 1325
PageValues Outliers: 2730
SpecialDay Outliers: 1249
```

Figure 12: Result of Outlier

Detecting Skewness

Why this technique was applied:

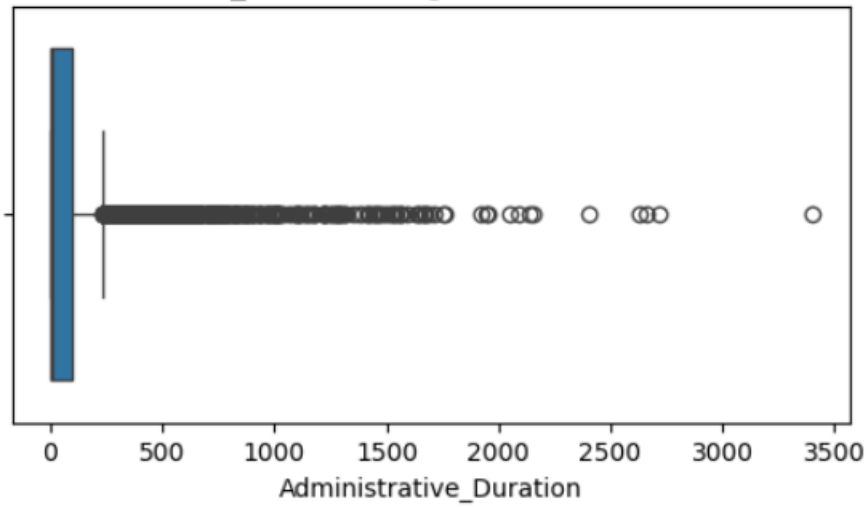
Skewness was checked to identify whether the numerical data was normally distributed or not. Highly skewed data can negatively affect statistical analysis and machine learning model performance. By detecting skewness, appropriate transformation techniques such as Log Transformation can be applied to normalize the data distribution and reduce the impact of extreme values. Skewness was checked to identify the distribution of data and detect whether the features were right-skewed or left-skewed. This helped in deciding whether transformation techniques were required to normalize the data.

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns
print("\nOutlier Type Detection")
for col in numerical_cols:
    skew_val = df[col].skew()
    if skew_val > 0.5:
        skew_type = "Right Skewed (Positive Skew)"
    elif skew_val < -0.5:
        skew_type = "Left Skewed (Negative Skew)"
    else:
        skew_type = "Approximately Symmetric"
    print(f"{col} --> Skewness: {skew_val:.2f} --> {skew_type}")
    plt.figure(figsize=(6,3))
    sns.boxplot(x=df[col])
    plt.title(f"{col} ({skew_type})")
    plt.show()
```

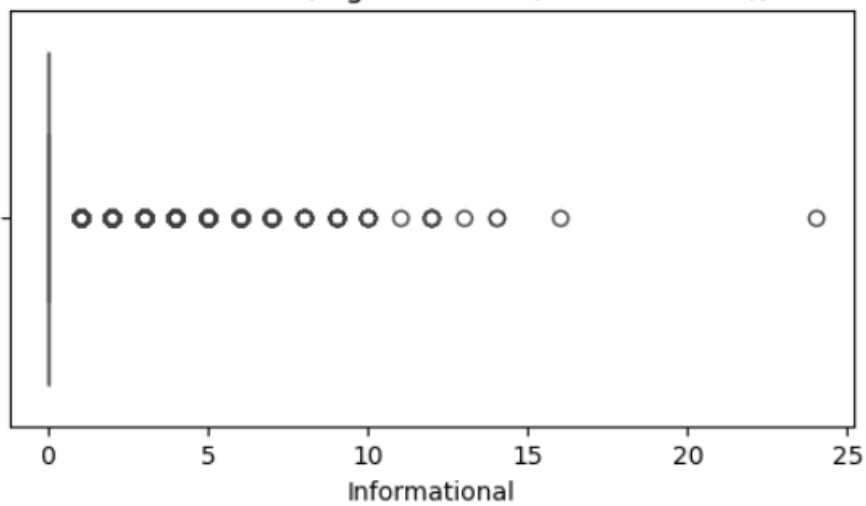
Administrative_Duration --> Skewness: 5.59 --> Right Skewed (Positive Skew)

Administrative_Duration (Right Skewed (Positive Skew))



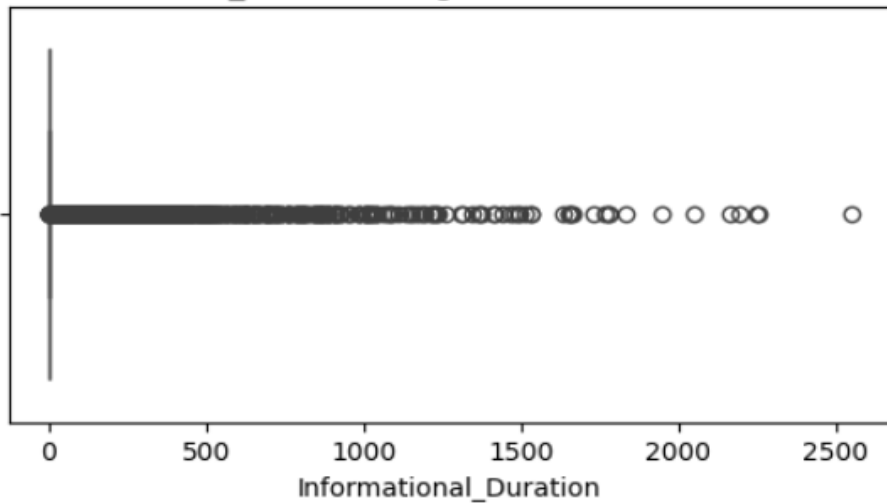
Informational --> Skewness: 4.01 --> Right Skewed (Positive Skew)

Informational (Right Skewed (Positive Skew))



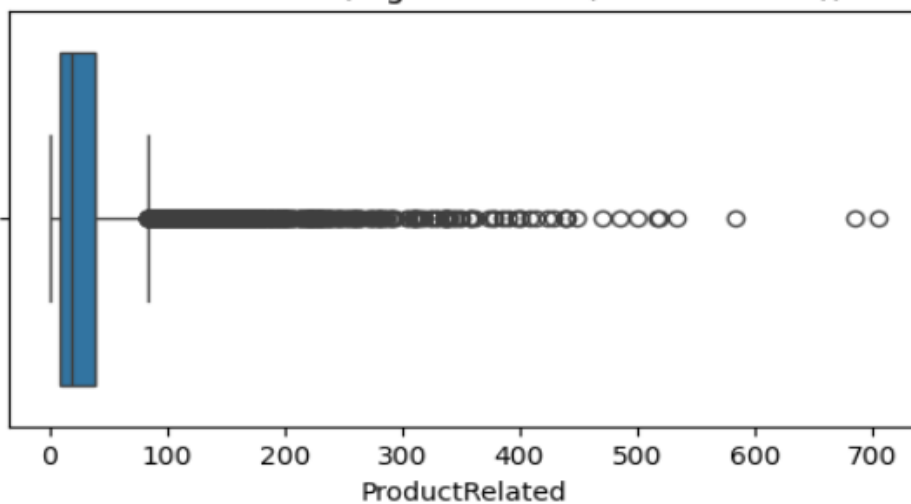
Informational_Duration --> Skewness: 7.54 --> Right Skewed (Positive Skew)

Informational_Duration (Right Skewed (Positive Skew))



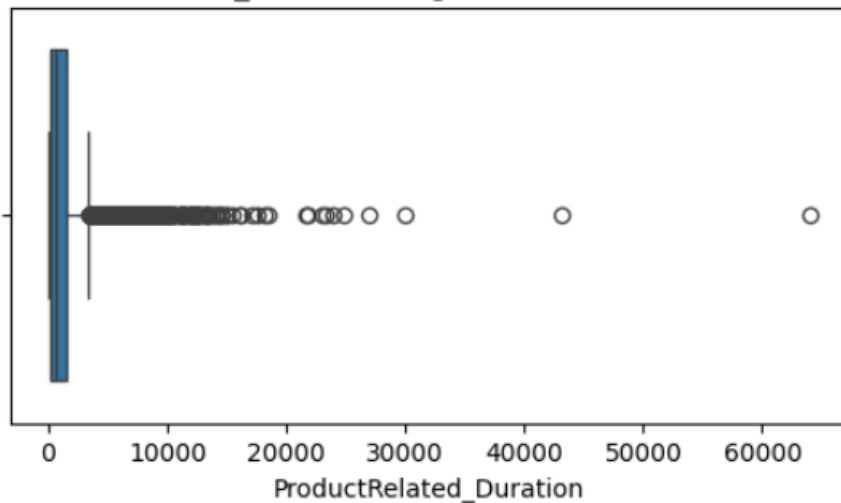
ProductRelated --> Skewness: 4.33 --> Right Skewed (Positive Skew)

ProductRelated (Right Skewed (Positive Skew))

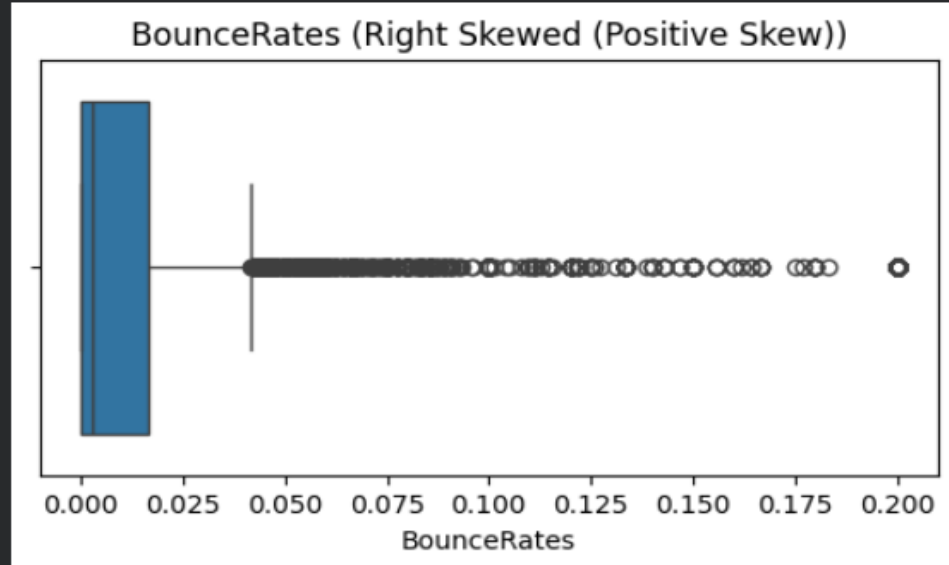


ProductRelated_Duration --> Skewness: 7.25 --> Right Skewed (Positive Skew)

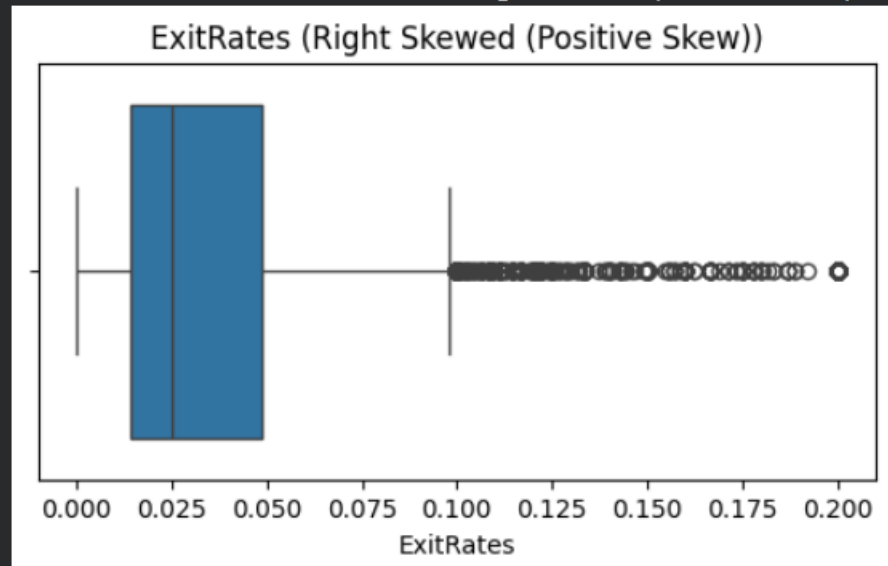
ProductRelated_Duration (Right Skewed (Positive Skew))

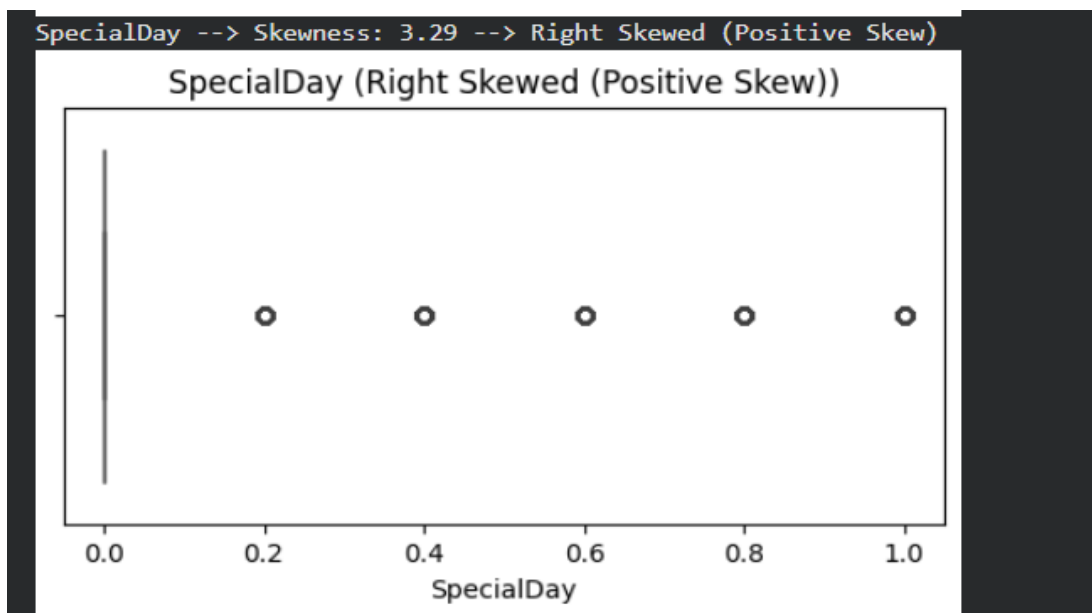
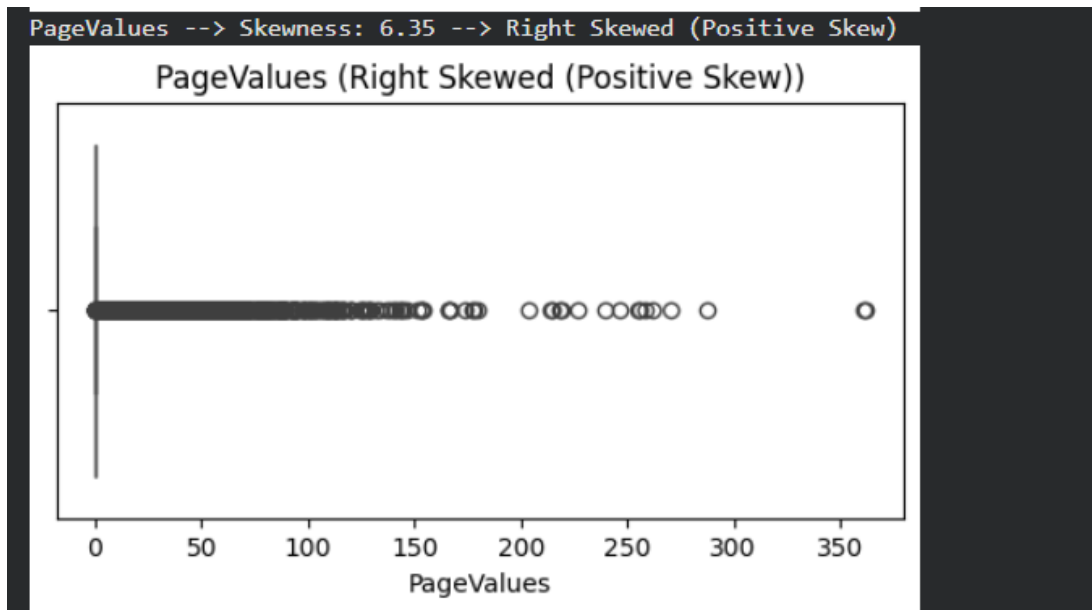


BounceRates --> Skewness: 3.16 --> Right Skewed (Positive Skew)



ExitRates --> Skewness: 2.23 --> Right Skewed (Positive Skew)





As the outliers are Right Skewed so there is a need to apply Log Transformation

Numerical columns were right-skewed So Log Transformation was required.

7. Log Transformation

Technique Applied: Log Transformation

Why this technique was applied:

Log transformation reduces skewness and minimizes the impact of extreme values.

Code:

```
transform_columns = ["Administrative", "Informational_Duration", "Informational", "ProductRelated", "ProductRelated_Duration", "BounceRates", "ExitRates", "PageValues"]
for transformation in transform_columns:
    df[transformation] = np.log1p(df[transformation])
for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(col, "Outliers After Log Transformation:", len(outliers))
```

Figure 13: Applying Log Transformation

Result:

```
*** Administrative Outliers After Log Transformation: 0
Administrative_Duration Outliers After Log Transformation: 1149
Informational Outliers After Log Transformation: 2631
Informational_Duration Outliers After Log Transformation: 2405
ProductRelated Outliers After Log Transformation: 542
ProductRelated_Duration Outliers After Log Transformation: 899
BounceRates Outliers After Log Transformation: 1415
ExitRates Outliers After Log Transformation: 971
PageValues Outliers After Log Transformation: 2730
SpecialDay Outliers After Log Transformation: 1249
```

Figure 14: Results after Log Transformation

After transformation, skewness and outliers were reduced. Data became more normalized.

8. Outlier Capping

Technique Applied: Capping using IQR Bounds

Why this technique was applied:

Some outliers still remained after log transformation.

Code:

```
numeric_columns = ["Administrative_Duration", "Informational", "Informational_Duration", "ProductRelated", "ProductRelated_Duration", "BounceRates", "ExitRates", "Page
for int_column in numeric_columns:
    quantile_1 = df[int_column].quantile(0.25)
    quantile_3 = df[int_column].quantile(0.75)
    calculated_IQR = quantile_3 - quantile_1
    lower_Bound = quantile_1 - (1.5 * calculated_IQR)
    upper_Bound = quantile_3 + (1.5 * calculated_IQR)
    df[int_column] = np.where(df[int_column] < lower_Bound, lower_Bound, df[int_column])
    df[int_column] = np.where(df[int_column] > upper_Bound, upper_Bound, df[int_column])
for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(col, "Outliers After Capping:", len(outliers))
```

Figure 15: Capping Code

Results:

```
*** Administrative Outliers After Capping: 0
Administrative_Duration Outliers After Capping: 0
Informational Outliers After Capping: 0
Informational_Duration Outliers After Capping: 0
ProductRelated Outliers After Capping: 0
ProductRelated_Duration Outliers After Capping: 0
BounceRates Outliers After Capping: 0
ExitRates Outliers After Capping: 0
PageValues Outliers After Capping: 0
SpecialDay Outliers After Capping: 0
```

Figure 16: Result Capping

Outliers were significantly reduced after capping. The dataset became more stable.

Feature Engineering

Technique Applied: Creating New Features

Why this technique was applied:

New features improve model understanding and increase predictive power.

New Feature:

1. Total Time Feature

Explanation:

This feature represents the total time spent by the customer.

2. Total Pages Feature

Explanation:

This feature calculates the total number of visited pages.

3. Engagement Ratio Feature

Explanation:

This feature measures customer engagement per page.

4. Per Page Feature

Explanation:

This feature calculates page value efficiency.

Results:

Code:

```
# 1. Total Time Spent
df['Total_Time'] = ( df['Administrative_Duration'] + df['Informational_Duration'] + df['ProductRelated_Duration'] )
# 2. Total Pages Visited
df['Total_Pages'] = ( df['Administrative'] + df['Informational'] + df['ProductRelated'] )
# 3. Engagement Ratio
df['Engagement_Ratio'] = df['Total_Time'] / (df['Total_Pages'] + 1)
# 4. Value Per Page
df['Value_Per_Page'] = df['PageValues'] / (df['ProductRelated'] + 1)
print("\nNew Features Added Successfully")
print(df[['Total_Time', 'Total_Pages', 'Engagement_Ratio', 'Value_Per_Page']].head())
df.info()
```

Figure 17: Feature Engineering

Results:

```
Data columns (total 22 columns):
#  Column  Non-Null Count  Dtype
---  -
0  Administrative  12205 non-null  float64
1  Administrative_Duration  12205 non-null  float64
2  Informational  12205 non-null  float64
3  Informational_Duration  12205 non-null  float64
4  ProductRelated  12205 non-null  float64
5  ProductRelated_Duration  12205 non-null  float64
6  BounceRates  12205 non-null  float64
7  ExitRates  12205 non-null  float64
8  PageValues  12205 non-null  float64
9  SpecialDay  12205 non-null  float64
10 Month  12205 non-null  object
11 OperatingSystems  12205 non-null  int64
12 Browser  12205 non-null  int64
13 Region  12205 non-null  int64
14 TrafficType  12205 non-null  int64
15 VisitorType  12205 non-null  object
16 Weekend  12205 non-null  bool
17 Revenue  12205 non-null  bool
18 Total_Time  12205 non-null  float64
19 Total_Pages  12205 non-null  float64
20 Engagement_Ratio  12205 non-null  float64
21 Value_Per_Page  12205 non-null  float64
dtypes: bool(2), float64(14), int64(4), object(2)
memory usage: 2.0+ MB
```

Figure 18: Datashape after Feature Engineering

New features were successfully added to the dataset. Feature engineering improved dataset quality and representation.

10. Advanced Feature Engineering and Interaction

Technique Applied: Polynomial Features

Why this technique was applied:

Polynomial Features were created to capture feature interactions and non-linear relationships between variables. This helps machine learning models understand complex patterns in the dataset. Moreover, there is no need to apply feature interaction separately because it has been done using polynomial features through its degree and biasness.

Code:

```
from sklearn.preprocessing import PolynomialFeatures
poly_cols = [
    'Total_Time',
    'Total_Pages',
    'Engagement_Ratio',
    'Value_Per_Page'
]
# Polynomial Feature Object
poly = PolynomialFeatures(degree=2, include_bias=False)
# Create Features
poly_features = poly.fit_transform(df[poly_cols])
# Feature Names
poly_feature_names = poly.get_feature_names_out(poly_cols)
# Convert to DataFrame
poly_df = pd.DataFrame(poly_features, columns=poly_feature_names)
# Remove original duplicate columns
poly_df = poly_df.drop(columns=poly_cols)
# Merge with original dataset
df = pd.concat([df.reset_index(drop=True), poly_df.reset_index(drop=True)], axis=1)
print("\nPolynomial Features Added Successfully")
df.info()
```

Figure 19: Implementation of Polynomial Features

Results:

```
...
Polynomial Features Added Successfully
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12205 entries, 0 to 12204
Data columns (total 32 columns):
```

Figure 20: Result of Polynomial Features

11. Binning

Technique Applied: Feature Binning

Why this technique was applied:

Binning was used to convert continuous numerical values into categorical groups such as Low, Medium, and High. This simplifies data interpretation and helps some machine learning models perform better.

Code:

```
# ProductRelated_Duration
df['ProductRelated_Duration_Bin'] = pd.cut(
    df['ProductRelated_Duration'],
    bins=3,
    labels=['Low', 'Medium', 'High']
)
# BounceRates
df['BounceRates_Bin'] = pd.cut(
    df['BounceRates'],
    bins=3,
    labels=['Low', 'Medium', 'High']
)

print(df[['ProductRelated_Duration_Bin',
           'BounceRates_Bin']].head())
```

Figure 21: Code of Binning

Results:

	ProductRelated_Duration_Bin	BounceRates_Bin
0	Low	High
1	Low	Low
2	Low	High
3	Low	High
4	Medium	Medium

Figure 22: Results of Binning

12. Feature Transformation: Scaling and Encoding

Technique Applied: Label Encoding and One-Hot Encoding

Why this technique was applied:

Machine learning models cannot process categorical text directly.

Label Encoding

Why this technique was applied:

Label Encoding was used to convert ordinal and binary categorical variables into numeric format.

One-Hot Encoding

One-Hot Encoding was used to convert nominal categorical values into separate binary columns so that machine learning models could process categorical data without assuming any order between categories.

```
# Encoding
# -----
binary_cols = ["Weekend" , "Revenue"]
nominal_cols = ["Month", "VisitorType"]
ordinal_cols = ["Browser" , "Region" , "OperatingSystems" , "TrafficType" , 'ProductRelated_Duration_Bin', 'BounceRates_Bin']
encoder = Encoder()
# Apply encoding on full dataset
X = encoder.label_encode(df, binary_cols)
X = encoder.onehot_encode(df, nominal_cols)
X = encoder.label_encode(df, ordinal_cols)
print("-----")
print("After Encoding Shape:", X.shape)
print("-----")
```

Figure 23: Implementation of Encoding

Technique Applied: Standard Scaling

Why this technique was applied:

The dataset contained numerical features with different ranges. Scaling was applied to standardize the data so that all features contribute equally.

Code:

```

# Scaling
# -----
numerical_cols = [
    "Administrative",
    "Administrative_Duration",
    "Informational",
    "Informational_Duration",
    "ProductRelated",
    "ProductRelated_Duration",
    "BounceRates",
    "ExitRates",
    "PageValues",
    "SpecialDay",
    "Total_Time",
    "Total_Pages",
    "Engagement_Ratio",
    "Value_Per_Page"
]
scaler = Scaler(method="standard")
# Fit + Transform on full dataset
X = scaler.fit_transform(df, numerical_cols)
X.describe()
print("-----")
print("After Scaling Shape:", df.shape)
print("-----")
X.head()

```

Figure 24: Implementation of Scaler

Results:

	Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration	BounceRates	ExitRates	PageValues	SpecialDay	...
0	-0.932479	-0.721911	0.0	0.0	-2.001350	-2.389594	2.147202	2.222963	0.0	0.0	...
1	-0.932479	-0.721911	0.0	0.0	-1.634840	-1.198896	-0.728527	2.153223	0.0	0.0	...
2	-0.932479	-0.721911	0.0	0.0	-2.001350	-2.389594	2.147202	2.222963	0.0	0.0	...
3	-0.932479	-0.721911	0.0	0.0	-1.634840	-2.389594	2.147202	2.222963	0.0	0.0	...
4	-0.932479	-0.721911	0.0	0.0	-0.460384	0.184735	0.649559	0.502485	0.0	0.0	...

5 rows x 34 columns

Figure 25: Figure of Final Dataset after Preprocessing

13. Statistical Analysis

Technique Applied: Descriptive Statistical Analysis

Why this technique was applied:

Statistical analysis was performed to understand the distribution, central tendency, and variability of the numerical features. Mean, median, mode, standard deviation, and variance helped in identifying data patterns, spread, and possible anomalies in the dataset.

Code:

```
numerical_cols = [
    "Administrative",
    "Administrative_Duration",
    "Informational",
    "Informational_Duration",
    "ProductRelated",
    "ProductRelated_Duration",
    "BounceRates",
    "ExitRates",
    "PageValues",
    "SpecialDay"
]
# Mean
print("Mean:")
print(df[numerical_cols].mean())
# Median
print("\nMedian:")
print(df[numerical_cols].median())
# Mode
print("\nMode:")
print(df[numerical_cols].mode().iloc[0])
# Standard Deviation
print("\nStandard Deviation:")
print(df[numerical_cols].std())
# Variance
print("\nVariance:")
print(df[numerical_cols].var())
# Minimum Values
print("\nMinimum Values:")
print(df[numerical_cols].min())
# Maximum Values
print("\nMaximum Values:")
print(df[numerical_cols].max())
# Complete Statistical Summary
print("\nStatistical Summary:")
print(df[numerical_cols].describe())
```

Figure 26: Statistical Analysis

Result:

Statistical Summary:				
	Administrative	Administrative_Duration	Informational	\
count	12205.000000	12205.000000	12205.0	
mean	0.805126	57.899459	0.0	
std	0.863461	80.206380	0.0	
min	0.000000	0.000000	0.0	
25%	0.000000	0.000000	0.0	
50%	0.693147	9.000000	0.0	
75%	1.609438	94.700000	0.0	
max	3.332205	236.750000	0.0	
	Informational_Duration	ProductRelated	ProductRelated_Duration	\
count	12205.0	12205.000000	12205.000000	
mean	0.0	2.907211	6.140398	
std	0.0	1.106330	1.639918	
min	0.0	0.000000	2.221821	
25%	0.0	2.197225	5.267858	
50%	0.0	2.944439	6.413365	
75%	0.0	3.663562	7.298550	
max	0.0	5.863067	10.344587	
	BounceRates	ExitRates	PageValues	SpecialDay
count	12205.000000	12205.000000	12205.0	12205.0
mean	0.010469	0.034629	0.0	0.0
std	0.014370	0.028183	0.0	0.0
min	0.000000	0.000000	0.0	0.0
25%	0.000000	0.014131	0.0	0.0
50%	0.002894	0.024693	0.0	0.0
75%	0.016529	0.047389	0.0	0.0
max	0.041323	0.097276	0.0	0.0

Figure 27: Summary of Statistical Analysis

The statistical analysis was performed to understand the distribution and variability of numerical features in the dataset. The results showed that all columns contained 12,205 records with no missing values. Measures such as mean, median, minimum, maximum, and standard deviation helped in identifying data spread and feature behavior. Some columns contained mostly zero values, while others showed high variation. This analysis helped in deciding further preprocessing techniques such as outlier handling, log transformation, scaling, and feature engineering.

Graphical Representation

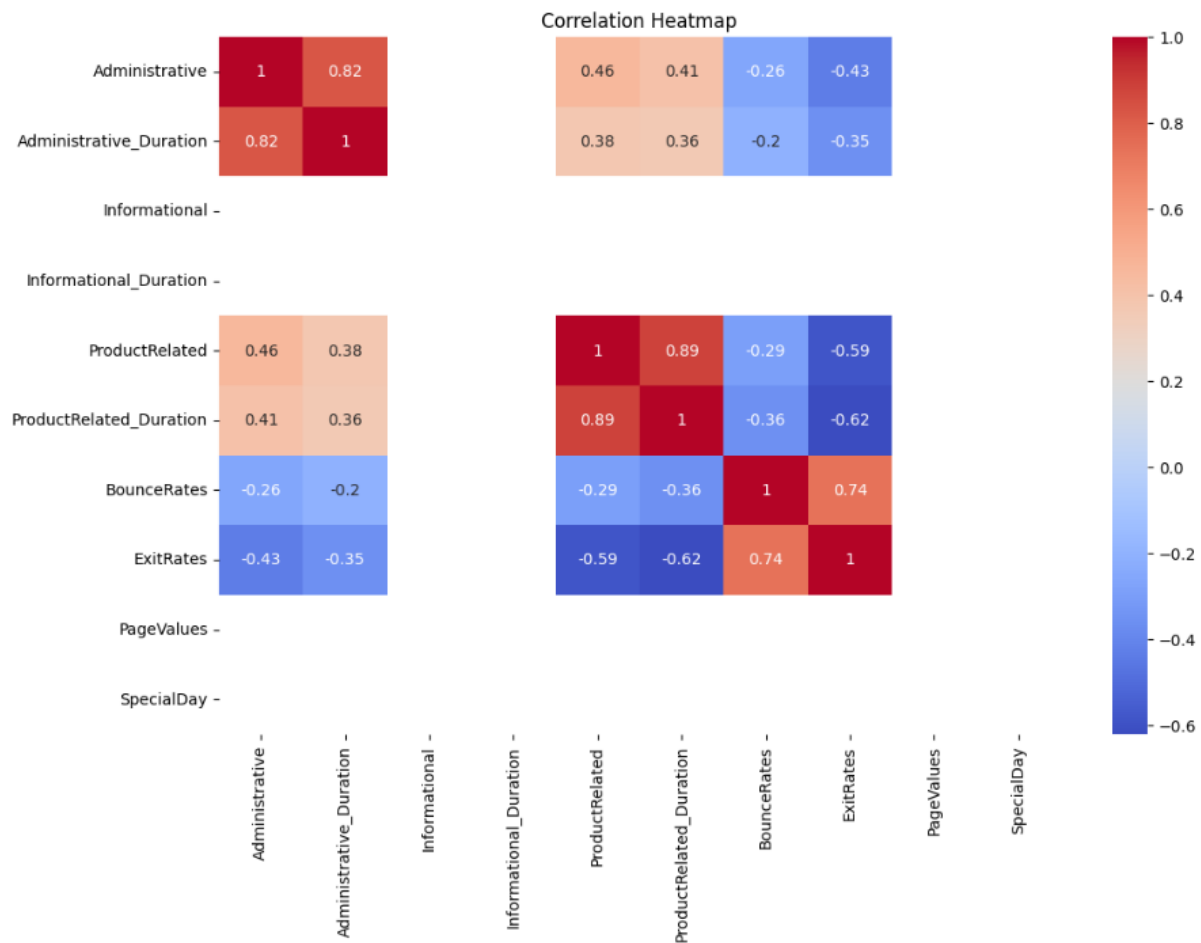


Figure 28: Heatmap after Statistical Analysis

The correlation heatmap was used to analyze the relationship between numerical features in the dataset. The results showed that Administrative and Administrative_Duration had a strong positive correlation of 0.82, while ProductRelated and ProductRelated_Duration had a very strong positive correlation of 0.89. BounceRates and ExitRates also showed a strong positive relationship of 0.74. Negative correlations were observed between ExitRates and ProductRelated_Duration (-0.62), which indicates that as product-related duration increases, exit rates decrease. Overall, the heatmap helped in identifying feature relationships, dependencies, and important patterns within the dataset.